

100 Days of Azure DevOps

Day 37 Handout — Batches replace instances while traffic still flows. Rollback is another movie — not a light switch

4 - Continuous Delivery · Personal lab only · Educational content · Not a sales pitch

Architecture A — where rolling actually lives (concepts; no AKS bill)

Platform	Knob	Mixed versions?
VMSS	upgradePolicy mode: Rolling; maxBatchInstancePercent; pauseTimeBetweenBatches	Yes, during the wave
AKS Deployment	strategy.rollingUpdate maxUnavailable / maxSurge	Yes, until the ReplicaSet drains
App Service	Instance replacement / restart — not the same as VMSS rolling	Usually a new zip on the fleet, not a designed N/N-1 wave
Slots / blue-green	Not rolling — idle world + flip	No mix on the live pointer
Canary	Not rolling — % of traffic, maybe one slot	Old + new by request share

Architecture B — pick with downtime, complexity, rollback

Strategy	Downtime / mix	Rollback
Rolling	Low downtime; N and N-1 live together	Slower — roll forward to old bits in batches
Blue-green	Near-zero cutover; no mix on the pointer	Fast swap if you paid for two worlds
Canary	Near-zero; small mix by design	Stop the ramp / 0% / flag off
In-place	Possible blip; one world	Redeploy previous artifact (you kept it)

Architecture C — pitfalls that make 'zero downtime' half-broken

Pitfall	What you see	Fix
Breaking API in the wave	20 minutes of mixed clients/servers	N-1 compatibility or do not roll
Breaking schema first	Old instances write, new ones migrate	Expand/contract. Rolling is not a migration tool
maxUnavailable: 100%	You invented an outage with extra YAML	Small batches; wait for healthy
Called App Service zip a rolling deploy	One restart, a blog title	Use the comparison table; stay honest

One-liner to remember

Rolling deploys change the tires while the car is moving — thrilling, and occasionally stupid if N and N-1 cannot share the road.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal repo only. No AKS, no VMSS spend required. The product today is a comparison you could defend in a design review.

- 1 Create docs/deploy-strategies-day37.md with a table: Strategy | Downtime | Complexity | Rollback | Mixed versions? for Rolling, Blue-green, Canary, In-place.
- 2 Fill every cell with something specific (swap, maxUnavailable, 5% ramp) — not 'low' / 'high' alone.
- 3 Write one scenario where rolling is wrong (breaking API or breaking schema) and which strategy you would pick instead.

- 4 Skim Learn: VMSS rolling upgrade OR Kubernetes rolling updates. Note one knob name in the doc (maxBatchInstancePercent or maxUnavailable).
- 5 Do not create AKS or a scale set for this lab. Cost is not a badge.
- 6 Cross-link docs/rollback.md (Day 35) and docs/canary-plan-day36.md so the three docs agree.

Comparison table (markdown you actually keep)

```
# docs/deploy-strategies-day37.md
# Strategy      | Downtime      | Mix N/N-1 | Rollback
# Rolling       | Low           | Yes       | Batches of the old bits (slow)
# Blue-green    | Near-zero     | No*       | Swap / flip pointer
# Canary        | Near-zero     | By %      | Stop ramp / 0%
# In-place      | Possible blip | Brief     | Redeploy kept artifact
# *live pointer is one color; green is idle
# Kubernetes sketch (do not apply today):
# spec.strategy.rollingUpdate: { maxUnavailable: 25%, maxSurge: 25% }
```

Done checklist

- Table exists with rollback + mixed-version columns
- Can explain why breaking changes forbid rolling
- No AKS/VMSS created for a metaphor
- Docs agree with Days 35–36

Tomorrow — Day 38

Approval gates and environments — checks live on the Environment, not in Slack.